Name: M. Farah Afrin

Sub code: CSA0923

Sub: Java Programming

Git Hub: https://github.com/farahafrinm/CSA0923-192511434

Assignment - 4

Introduction

Event-driven programming is a programming approach in which the flow of execution is controlled by events generated through user interaction or by the system. In Java GUI applications, events such as button clicks, key presses, mouse clicks and window-closing actions are handled through listener interfaces and adapter classes.

This assignment examines the design and implementation of event-driven interfaces using AWT and Swing. It covers event delegation, component hierarchy, layout managers, listener implementation styles, repainting, validation and GUI–JDBC integration. The solutions emphasise separation of interface code from application logic, reliable event handling and responsive user interaction.

The six tasks are organised in the same order as the assignment brief. Each task contains the written analysis and design first, followed by the relevant Java source code where implementation is required.

TASK 1 – GUI DESIGN REVIEW (6 MARKS)

1.1 Diagnose the Design

The given fee-payment program places input handling, database access, waiting logic and output inside one actionPerformed() method. This creates a tightly coupled design and makes the interface difficult to test and maintain.

No.	Design deficiency	Effect or possible failure
1	User-interface code and payment logic are combined.	A change in payment rules requires changes to the GUI listener. Independent testing of the payment logic becomes difficult.
2	The action command is compared using ==.	The operator compares object references rather than string contents. A matching command may not be recognised reliably; equals() should be used.
3	Input is parsed without checking for empty or invalid values.	An empty or non-numeric amount causes NumberFormatException. Zero or negative

		payments may also be accepted without a business rule.
4	Database operations run inside the event-dispatch thread.	A slow database response blocks the GUI event thread and makes the window appear frozen.
5	Thread.sleep(3000) is called in the event handler.	The interface cannot respond or repaint during the delay. Sleeping does not provide genuine bank confirmation.
6	The program prints “Paid” only to the console.	The user receives no clear on-screen confirmation, failure message or transaction reference.
7	The Receipt button is not handled separately.	The else branch clears the fields for every action other than Pay, so Receipt does not produce a receipt.
8	SQL is constructed by concatenating user input.	The query is vulnerable to SQL injection and can fail when input contains special characters.
9	The database connection and statement are not closed safely.	Database resources may remain allocated. try-with-resources should manage them.
10	There is no window-closing handler or exception handling.	The program has no controlled shutdown or user-friendly response to database errors and checked exceptions.

1.2 Repair the Design

The repaired design separates the program into two principal classes:

- FeeWindow – the presentation layer. It collects the register number and amount, registers the button listeners and displays the payment result.
- FeeService – the service layer. It validates the input, applies payment rules and performs the database insertion.
- JDBC database – stores the payment record. PreparedStatement is used to bind values safely, and try-with-resources closes database objects.

The Pay, Clear and Receipt buttons have independent actions. The Pay action calls FeeService rather than embedding SQL in the window. The Clear action resets the fields. The Receipt action provides a separate receipt-related response. The window listener controls closing.

1.3 Design Defence

The redesigned program is easier to test because FeeService can be tested independently with valid, empty and invalid input without opening a window. FeeWindow is responsible only for collecting values and presenting results. Payment rules and database operations are centralised, so they can be modified without rewriting the interface. PreparedStatement reduces SQL-injection risk, while try-with-resources closes JDBC resources reliably. Short event handlers are easier to read, debug and extend.

Alternative Concept and Complexity Analysis

With one listener and an if–else chain, the application may need to compare the action command against as many as k button commands. The worst-case application-level selection cost is O(k). If each button has its own listener object or Swing Action, the event source is already associated with its handler, so the application's own selection step is O(1). A shared listener uses O(1) listener objects, whereas k independent listener objects require O(k) space. The independent-listener design is clearer when each button has different behaviour. The actual internal dispatch cost of the GUI toolkit is implementation-dependent.

Java Implementation – Task 1.2

The following two source files implement the redesigned fee-payment window and its service class.

Java Source Code – FeeWindow.java

File name: FeeWindow.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class FeeWindow extends JFrame {
    private final JTextField regNo = new JTextField(15);
    private final JTextField amount = new JTextField(15);
    private final FeeService feeService = new FeeService();

    public FeeWindow() {
        super("Student Fee Payment");
        setLayout(new BorderLayout(10, 10));

        JPanel fields = new JPanel(new GridLayout(2, 2, 8, 8));
        fields.setBorder(BorderFactory.createEmptyBorder(15, 15, 5, 15));
        fields.add(new JLabel("Register number:"));
        fields.add(regNo);
        fields.add(new JLabel("Amount (INR):"));
        fields.add(amount);

        JButton pay = new JButton("Pay");
        JButton clear = new JButton("Clear");
        JButton receipt = new JButton("Receipt");
```

```
        JPanel actions = new JPanel(new FlowLayout(FlowLayout.CENTER));
        actions.add(pay);
        actions.add(clear);
        actions.add(receipt);

        add(fields, BorderLayout.CENTER);
        add(actions, BorderLayout.SOUTH);

        pay.addActionListener(e -> processPayment());
        clear.addActionListener(e -> clearFields());
        receipt.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Receipt is available after a successful payment.",
                "Receipt", JOptionPane.INFORMATION_MESSAGE));

        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) {
                int choice = JOptionPane.showConfirmDialog(FeeWindow.this,
                    "Close the fee payment window?", "Confirm",
                    JOptionPane.YES_NO_OPTION);
                if (choice == JOptionPane.YES_OPTION) dispose();
            }
        });
        pack();
        setLocationRelativeTo(null);
    }

    private void processPayment() {
        final String id = regNo.getText();
        final String fee = amount.getText();
        new SwingWorker<String, Void>() {
            @Override protected String doInBackground() {
                return feeService.payFee(id, fee);
            }
            @Override protected void done() {
                try {
                    String message = get();
                    JOptionPane.showMessageDialog(FeeWindow.this, message,
                        "Payment status", message.startsWith("Payment recorded")
                            ? JOptionPane.INFORMATION_MESSAGE :
JOptionPane.WARNING_MESSAGE);
                } catch (Exception ex) {
                    JOptionPane.showMessageDialog(FeeWindow.this,
```

```
                "Payment failed: " + ex.getMessage());
            }
        }
    }.execute();
}

private void clearFields() {
    regNo.setText("");
    amount.setText("");
    regNo.requestFocus();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new FeeWindow().setVisible(true));
}
}
```

Output:

```
package q1;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class FeeWindow extends JFrame {
    private final JTextField regNo = new JTextField(15);
    private final JTextField amount = new JTextField(15);
    private final FeeService feeService = new FeeService();

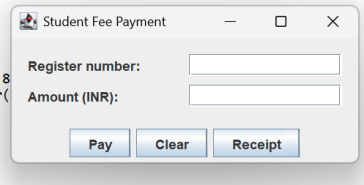
    public FeeWindow() {
        super("Student Fee Payment");
        setLayout(new BorderLayout(10, 10));

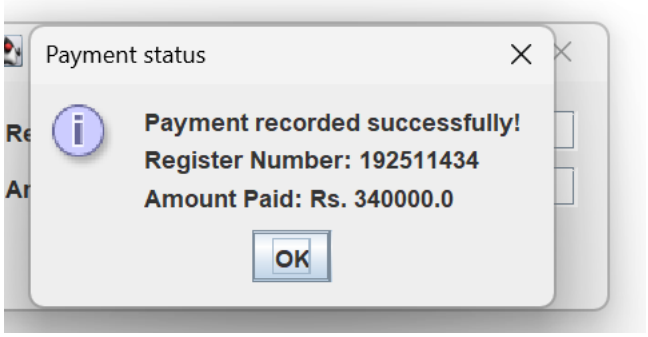
        JPanel fields = new JPanel(new GridLayout(2, 2, 8
        fields.setBorder(BorderFactory.createEmptyBorder(
        fields.add(new JLabel("Register number:"));
        fields.add(regNo);
        fields.add(new JLabel("Amount (INR):"));
        fields.add(amount);

        JButton pay = new JButton("Pay");
        JButton clear = new JButton("Clear");
        JButton receipt = new JButton("Receipt");
        JPanel actions = new JPanel(new FlowLayout(FlowLayout.CENTER));
        actions.add(pay);
        actions.add(clear);
        actions.add(receipt);

        add(fields, BorderLayout.CENTER);
        add(actions, BorderLayout.SOUTH);

        pay.addActionListener(e -> processPayment());
        clear.addActionListener(e -> clearFields());
```





Java Source Code – FeeService.java

File name: FeeService.java

```
import java.sql.Connection;
```

```
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

/**
 * Business layer for fee payments.
 * The GUI does not contain SQL or payment rules.
 */
public class FeeService {
    private static final String URL = "jdbc:mysql://localhost:3306/college_db";
    private static final String USER = "root";
    private static final String PASSWORD = "YOUR_MYSQL_PASSWORD";

    public String payFee(String registerNumber, String amountText) {
        if (registerNumber == null || registerNumber.trim().isEmpty()) {
            return "Enter the register number.";
        }

        double amount;
        try {
            amount = Double.parseDouble(amountText.trim());
        } catch (NumberFormatException ex) {
            return "Enter a valid numeric amount.";
        }

        if (!Double.isFinite(amount) || amount <= 0) {
            return "Amount must be greater than zero.";
        }

        String sql = "INSERT INTO fees(register_no, amount) VALUES (?, ?)";
        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
             PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, registerNumber.trim());
            statement.setDouble(2, amount);
            statement.executeUpdate();
            return "Payment recorded successfully.";
        } catch (SQLException ex) {
            return "Payment could not be recorded: " + ex.getMessage();
        }
    }
}
```

TASK 2 – FROM REQUIREMENT TO EVENT-DRIVEN DESIGN (8 MARKS)

Hospital Outpatient Appointment Desk

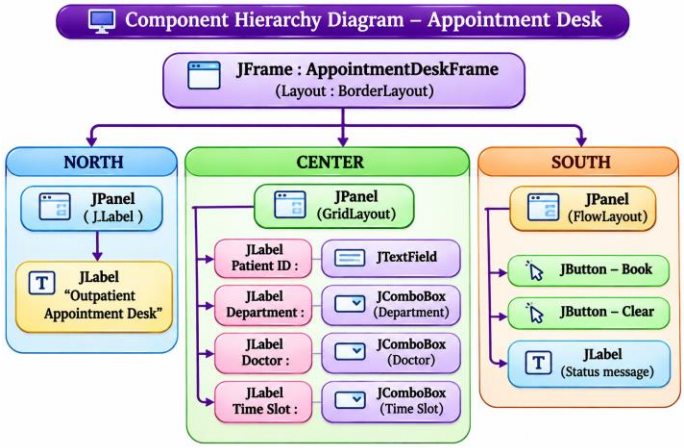
The outpatient appointment desk requires a form that allows a receptionist to enter a patient ID, select a department, choose a doctor belonging to that department, select a time slot and book an appointment. The interface must provide clear confirmation or error feedback and must request confirmation before closing when a booking is in progress.

2.1 Model the Events

Event source	Event object	Listener interface	Action performed
Patient ID JTextField	ActionEvent	ActionListener	Optionally validate or retrieve the patient when Enter is pressed.
Department JComboBox	ItemEvent / ActionEvent	ItemListener / ActionListener	Clear dependent selections and load doctors for the selected department.
Doctor JComboBox	ActionEvent	ActionListener	Load or refresh the time slots available for the selected doctor.
Time-slot JComboBox	ActionEvent	ActionListener	Record the selected appointment time.
Book JButton	ActionEvent	ActionListener	Validate details, check slot availability and save the appointment.
Window close control	WindowEvent	WindowListener / WindowAdapter	Ask for confirmation if a booking is in progress; close only after confirmation.

2.2 Component Hierarchy

The proposed hierarchy uses a JFrame as the main window and nested JPanel containers to group related controls.



JFrame provides the top-level window. JPanel organises related components. JTextField accepts the patient ID, JComboBox restricts selections to available options, JButton initiates actions and JLabel displays feedback.

2.3 Handler Logic – Algorithmic Pseudocode

Algorithm: REFRESH_DOCTORS

1. Read the department selected in departmentCombo.
2. Remove the existing entries from doctorCombo and timeSlotCombo.
3. If no department is selected, display "Select a department" and terminate the algorithm.
4. Retrieve the doctors associated with the selected department from the department-to-doctor mapping.
5. Add each retrieved doctor to doctorCombo.
6. If the doctor list is empty, display "No doctors available for this department".
7. Otherwise, select the first doctor and refresh the available time slots for that doctor.
8. Terminate the algorithm.

Algorithm: BOOK_APPOINTMENT

9. Read the patient ID, department, doctor and time slot from the form.
10. If any required field is empty or unselected, display an appropriate validation message and terminate.
11. Validate the patient ID and verify that the patient record exists.
12. Check whether the selected doctor and time slot are available.
13. If the slot is unavailable, display "Selected slot is no longer available" and terminate.
14. Display the booking details and request confirmation from the receptionist.
15. If the receptionist selects No, cancel the operation without saving.
16. Save the appointment through the service and data-access layers.
17. Display the appointment confirmation number and update the booking status.
18. Terminate the algorithm.

2.4 Trace the Event Sequence

The window-opening sequence is: main() → SwingUtilities.invokeLater(...) → AppointmentDeskFrame constructor → create components and models → register listeners → pack() or setSize() → setVisible(true). When the department selection changes, the JComboBox generates an event and the

registered listener refreshes the doctor list. When Book is clicked, the ActionListener validates the details and initiates the booking operation.

When the user selects the close control, windowClosing(WindowEvent) is invoked. If a booking is in progress, the handler displays a confirmation dialog. If the user confirms, dispose() closes the window; otherwise, the window remains open. The windowClosed event may occur after disposal.

An applet has a lifecycle controlled by its host environment. init() performs initialisation, start() begins or resumes activity, paint() draws the applet, stop() pauses activity and destroy() releases resources before removal. A JFrame does not use these applet lifecycle methods; its constructor, visibility, event callbacks and dispose() method control its lifecycle.

Alternative Concept and Complexity Analysis

If the doctor list is fetched from the database every time the department changes, each selection causes a database round trip and query. The elapsed time depends on connection/query overhead and the number of returned doctors. A cached Map<String, List<Doctor>> provides expected O(1) lookup of a department's list, followed by O(m_d) work to populate the doctor selector, where m_d is the number of doctors in that department. For d departments and m doctors in total, the map requires O(d + m) space. Database-on-demand reduces the amount of cached data but increases repeated query latency and database load. Caching improves responsiveness but requires a refresh strategy when doctor information changes.

TASK 3 – FROM SKETCH TO LAYOUT (9 MARKS)

Campus Library Search Screen

The library search screen contains a title bar, a search row, a results area, navigation and reservation buttons, and a status message. The central results area must grow when the window is resized, while the search row and action buttons retain their preferred sizes.

3.1 Choose the Layout Managers

Container / region	Layout manager	Reason for selection
Main JFrame	BorderLayout	Separates the top, centre and bottom regions. The centre receives additional space when the frame grows.
Title panel	FlowLayout	Centres the title without stretching the title component across the full window.
Search row	FlowLayout(LEFT)	Places the text field, category Choice/JComboBox and Search button in one row at their preferred sizes.
Results region	BorderLayout with JScrollPane	The JTable expands in the centre and provides scrolling when the records exceed the visible area.
Bottom action region	BorderLayout with FlowLayout	Keeps Prev, Next and Reserve together while allowing the status message to occupy its own row.

3.2 Nested Container Structure

JFrame (BorderLayout)

NORTH: Title JPanel (FlowLayout)

PAGE_START: Search JPanel (FlowLayout.LEFT)

JLabel – Search

JTextField – Search text

JComboBox – Category

JButton – Search

CENTER: JScrollPane

JTable – Search results

SOUTH: Bottom JPanel (BorderLayout)

NORTH: Action JPanel (FlowLayout.RIGHT) – Prev, Next, Reserve

SOUTH: JLabel – Status message

3.3 Implementation

The Swing implementation below uses a BorderLayout-based frame, a FlowLayout search row, a JTable inside JScrollPane and a bottom action/status panel. Demonstration records are used so the interface can be executed without a database. The search method filters the sample records according to the entered title and selected category.

Java Source Code – LibrarySearchScreen.java

File name: LibrarySearchScreen.java

```
package q2;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

public class LibrarySearchScreen extends JFrame {

    private static final long serialVersionUID = 1L;

    private final JTextField searchField = new JTextField(22);

    private final JComboBox<String> category =
            new JComboBox<>(new String[]{
                "All", "Programming", "Electronics", "AI"
            });

    private final DefaultTableModel model =
            new DefaultTableModel(
                new String[]{"Book ID", "Title", "Category", "Availability"}, 0
            );

    private final JTable results = new JTable(model);
```

```
private final JLabel status =
        new JLabel("Ready. Enter a title or choose a category.");

public LibrarySearchScreen() {

    super("Campus Library - Smart Search");

    setLayout(new BorderLayout(8, 8));

    JLabel title = new JLabel(
            "CAMPUS LIBRARY SEARCH", SwingConstants.CENTER);

    title.setFont(new Font("SansSerif", Font.BOLD, 20));

    title.setBorder(
            BorderFactory.createEmptyBorder(12, 5, 12, 5));

    JPanel searchPanel =
            new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 8));

    searchPanel.add(new JLabel("Search:"));
    searchPanel.add(searchField);
    searchPanel.add(category);

    JButton search = new JButton("Search");
    searchPanel.add(search);

    // Header panel
    JPanel header = new JPanel(new BorderLayout());

    JPanel top = new JPanel(new BorderLayout());

    top.add(title, BorderLayout.NORTH);
```

```
header.add(top, BorderLayout.CENTER);
header.add(searchPanel, BorderLayout.SOUTH);

add(header, BorderLayout.NORTH);

// Results table
results.setFillsViewportHeight(true);

add(new JScrollPane(results), BorderLayout.CENTER);

// Bottom panel
JPanel bottom = new JPanel(new BorderLayout());

JPanel buttons =
        new JPanel(new FlowLayout(FlowLayout.RIGHT));

JButton prev = new JButton("Prev");
JButton next = new JButton("Next");
JButton reserve = new JButton("Reserve");

buttons.add(prev);
buttons.add(next);
buttons.add(reserve);

bottom.add(buttons, BorderLayout.NORTH);

status.setBorder(
        BorderFactory.createEmptyBorder(6, 10, 6, 10));

bottom.add(status, BorderLayout.SOUTH);

add(bottom, BorderLayout.SOUTH);
```

```
    // Button actions
    search.addActionListener(e -> performSearch());

    searchField.addActionListener(e -> performSearch());

    prev.addActionListener(e ->
            status.setText("Previous page requested."));

    next.addActionListener(e ->
            status.setText("Next page requested."));

    reserve.addActionListener(e -> reserveSelectedBook());

    // Window closing
    addWindowListener(new WindowAdapter() {

        @Override
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);

    setMinimumSize(new Dimension(650, 420));

    setSize(850, 560);

    setLocationRelativeTo(null);
}

// Search books
```

```
private void performSearch() {

    model.setRowCount(0);

    String query = searchField.getText().trim();

    String selected = (String) category.getSelectedItem();

    // Demonstration book records
    Object[][] books = {

        {"B101", "Java: The Complete Reference",
                "Programming", "Available"},

        {"B102", "Digital Electronics",
                "Electronics", "Available"},

        {"B103", "Artificial Intelligence: A Modern Approach",
                "AI", "Issued"},

        {"B104", "Effective Java",
                "Programming", "Available"}
    };

    int shown = 0;

    for (Object[] book : books) {

        boolean categoryMatch =
                "All".equals(selected) || selected.equals(book[2]);

        boolean textMatch =
                query.isEmpty() ||
```

```
                book[1].toString().toLowerCase()
                .contains(query.toLowerCase());

        if (categoryMatch && textMatch) {

            model.addRow(book);

            shown++;
        }
    }

    status.setText(shown + " matching book(s) displayed.");
}

// Reserve selected book
private void reserveSelectedBook() {

    int row = results.getSelectedRow();

    if (row < 0) {

        JOptionPane.showMessageDialog(
                this,
                "Select a book before reserving.");

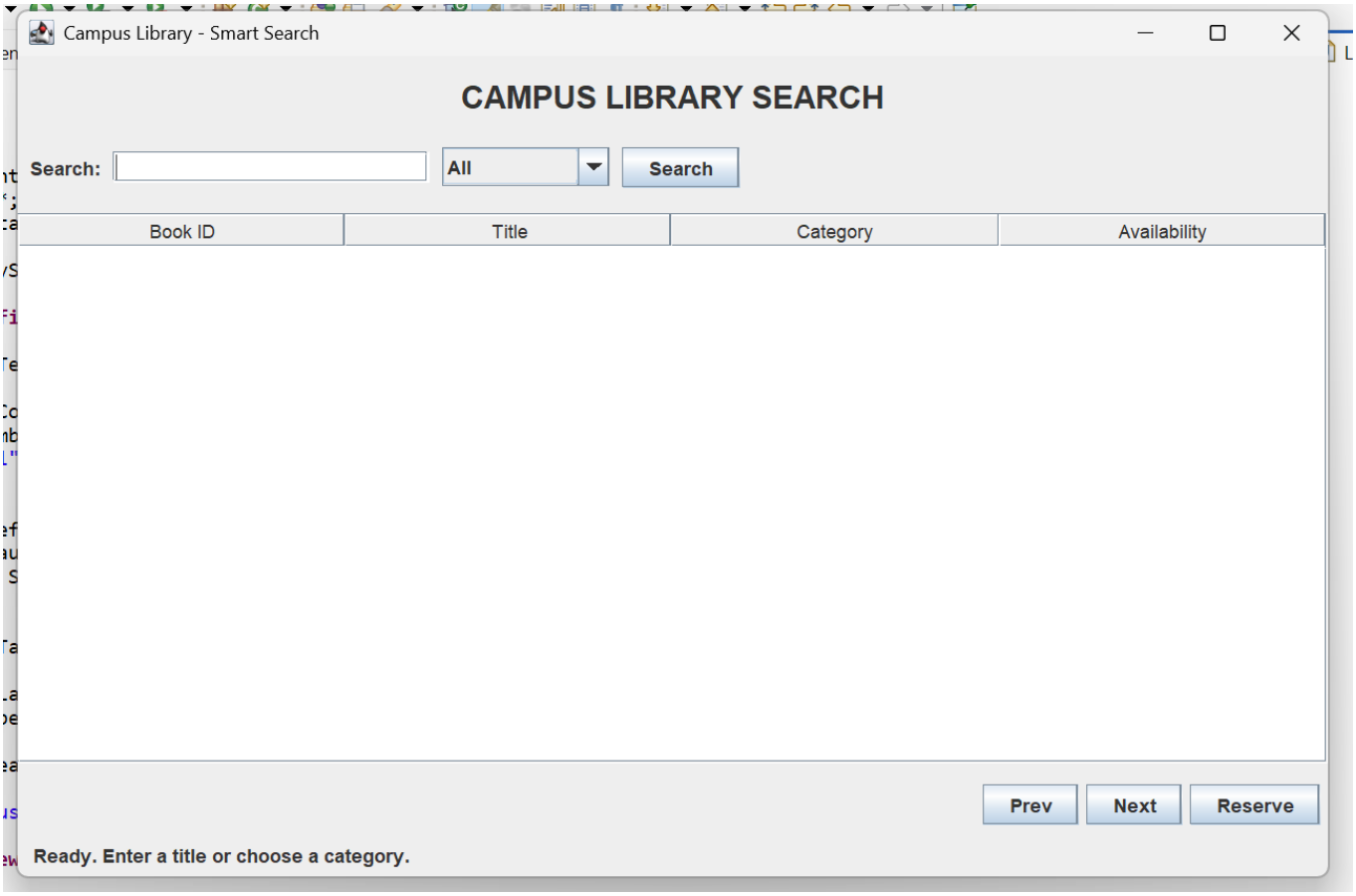
        return;
    }

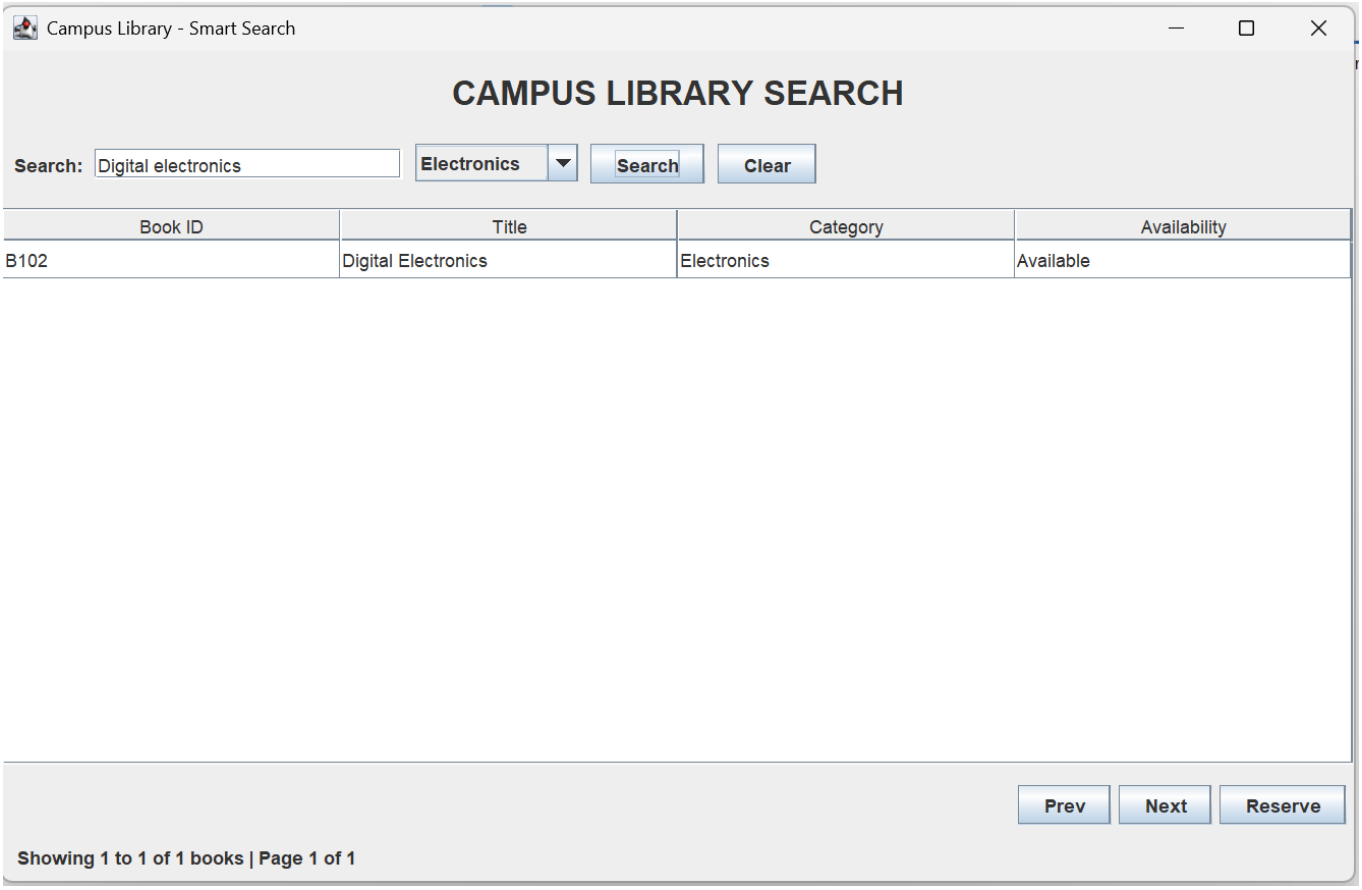
    JOptionPane.showMessageDialog(
            this,
            "Reservation request recorded for "
            + model.getValueAt(row, 1) + ".");
}
```

```
    // Main method
    public static void main(String[] args) {

        SwingUtilities.invokeLater(() ->
                new LibrarySearchScreen().setVisible(true));
    }
}
```

Output:





3.4 Resizing Behaviour and Test Cases

Test	Window size	Expected behaviour
R1	650 × 420	The controls remain visible at the minimum size. The results table occupies the remaining centre area and may show scroll bars.
R2	850 × 560	The results area expands while the search row and bottom buttons retain their preferred dimensions.
R3	1100 × 700	The table becomes wider and taller. The search controls and action buttons remain grouped and aligned.

Alternative Concept and Complexity Analysis

GridBagLayout can create a flexible screen with fewer nested panels, but it requires constraints such as gridx, gridy, weightx, weighty and fill. Correct weights allow the results area to expand; incorrect constraints may cause uneven resizing. Absolute positioning with setLayout(null) and setBounds() gives precise initial placement but does not adapt naturally to different window sizes, display scaling or font settings. A typical layout pass processes the components, giving work broadly proportional to O(c), where c is the number of components, although constraint-based layouts have higher constant overhead. Nested panels require O(p) additional container references for p panels but make the design easier to maintain. Absolute positioning may use fewer containers but requires manual resizing logic.

TASK 4 – EVENT-HANDLING FORENSICS (7 MARKS)

The drawing pad is expected to draw a dot at each mouse click and display the total number of clicks. The supplied implementation contains a callback naming error and uses direct screen drawing rather than retaining drawing data.

4.1 Locate the Defect

The method is written as mouseclicked(MouseEvent e), whereas the MouseListener callback is case-sensitive and must be named mouseClicked(MouseEvent e). Java treats these as different method names. Therefore, the adapter's empty mouseClicked() method is invoked and the incorrectly named method is never called. The label also remains unchanged because the original code does not call status.setText() after a click.

4.2 Construct a Failing Interaction

After correcting the callback name, click several positions in the drawing area. Then minimise the window and restore it. The dots may disappear because getGraphics() draws directly onto the current screen surface. The points are not stored in the component's state. When the window is restored, the operating system requests repainting and the paint() method redraws the component. Since the original program has not retained the clicked coordinates, the dots cannot be recreated.

4.3 Repair the Program

The repaired implementation stores each click as a Point in an ArrayList<Point>. The click handler increments the effective count by adding a point, updates the status label and calls repaint(). The paint() method iterates over all stored points and draws a 6 × 6 filled circle centred at each point. This makes the drawing persistent across repaint operations.

Java Source Code – DrawPad.java

File name: DrawPad.java

```
import java.awt.BorderLayout;
import java.awt.Frame;
import java.awt.Graphics;
import java.awt.Label;
import java.awt.Point;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.ArrayList;
import java.util.List;

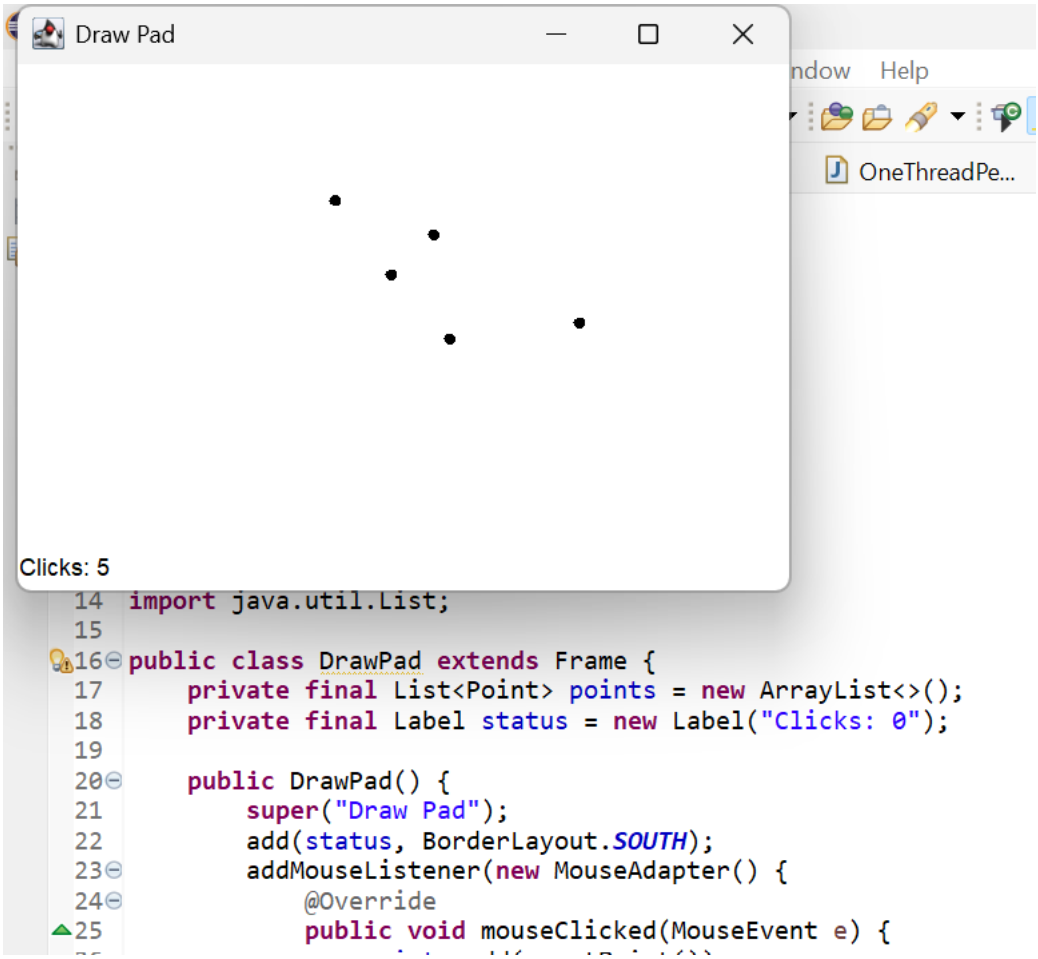
public class DrawPad extends Frame {
    private final List<Point> points = new ArrayList<>();
    private final Label status = new Label("Clicks: 0");

    public DrawPad() {
```

```
        super("Draw Pad");
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                points.add(e.getPoint());
                status.setText("Clicks: " + points.size());
                repaint();
            }
        });
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setSize(400, 300);
    }

    @Override
    public void paint(Graphics g) {
        super.paint(g);
        for (Point point : points) {
            g.fillOval(point.x - 3, point.y - 3, 6, 6);
        }
    }

    public static void main(String[] args) {
        new DrawPad().setVisible(true);
    }
}
```



4.4 Prevent Recurrence

The @Override annotation instructs the compiler to verify that the method overrides a method declared in a superclass or implemented interface. If @Override is placed above the incorrectly spelt mouseclicked() method, compilation fails because no inherited method with that signature exists. This catches the callback error before execution.

Complete the Code – TODO 1 and TODO 2

TODO 1 – Correct mouse callback:

```
@Override
public void mouseClicked(MouseEvent e) {
    points.add(e.getPoint());
    status.setText("Clicks: " + points.size());
    repaint();
}
```

TODO 2 – Redraw the stored points:

```
for (Point point : points) {
    g.fillOval(point.x - 3, point.y - 3, 6, 6);
}
```

For the negative test, misspell mouseClicked as mouseclicked while retaining @Override. The compiler should report that the method does not override or implement a method from a supertype. If the annotation is removed as well, the class may compile, but the event callback will still not be invoked.

Alternative Concept and Complexity Analysis

With an ArrayList of n points, each full repaint draws n circles, so the drawing work is O(n), and the point list requires O(n) space. An off-screen BufferedImage stores the drawing as pixels. A repaint can copy the image to the screen; a full-image copy is proportional to O(w × h), where w and h are the image dimensions. The image requires O(w × h) pixel storage. The point-list method is compact when the number of points is small and supports redrawing at different scales. The image method can make repeated rendering efficient for dense drawings but requires image initialisation, resizing and graphics-resource management.

TASK 5 – SAME SCREEN, DIFFERENT EVENT-HANDLING STYLES (7 MARKS)

The currency converter accepts an amount in Indian Rupees and converts it into a selected target currency. The Convert button and Enter key in the amount field must trigger the same conversion operation. A dialog displays the result or an input error.

5.1 Four Implementations

The four implementations below demonstrate the same user action using different event-handling styles: the frame implements ActionListener, a named inner class, an anonymous inner class and a lambda expression. Each version also uses WindowAdapter for window closing.

(a) Frame Implements ActionListener

Java Source Code – ConverterFrameListener.java

File name: ConverterFrameListener.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ConverterFrameListener extends JFrame implements ActionListener {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton button = new JButton("Convert");

    public ConverterFrameListener() {
        super("Converter – Frame Listener");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(button); add(result);
        button.addActionListener(this);
        amount.addActionListener(this);
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        pack(); setLocationRelativeTo(null);
    }

    @Override public void actionPerformed(ActionEvent e) { convert(); }

    private void convert() {
        try {
            double value = Double.parseDouble(amount.getText().trim());
            if (!Double.isFinite(value) || value < 0) throw new NumberFormatException();
```

```
            String c = (String) target.getSelectedItem();
            double rate = c.equals("USD") ? 0.012 : 0.011;
            String answer = String.format("%.2f %s", value * rate, c);
            result.setText(answer);
            JOptionPane.showMessageDialog(this, answer);
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
            JOptionPane.showMessageDialog(this, "Invalid amount");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ConverterFrameListener().setVisible(true));
    }
}
```

```
er extends JFrame implements ActionListener {
nt = new JTextField(10);
g> target = new JComboBox<>(new String[]{"USD", "EUR"});
 new JLabel(" ");
= new JButton("Convert");

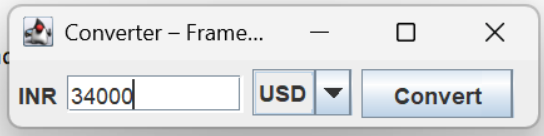
() {
Listener");
));
dd(amount); add(target); ad
this);
this);
dowAdapter() {
 windowClosing(WindowEvent e) { dispose(); }

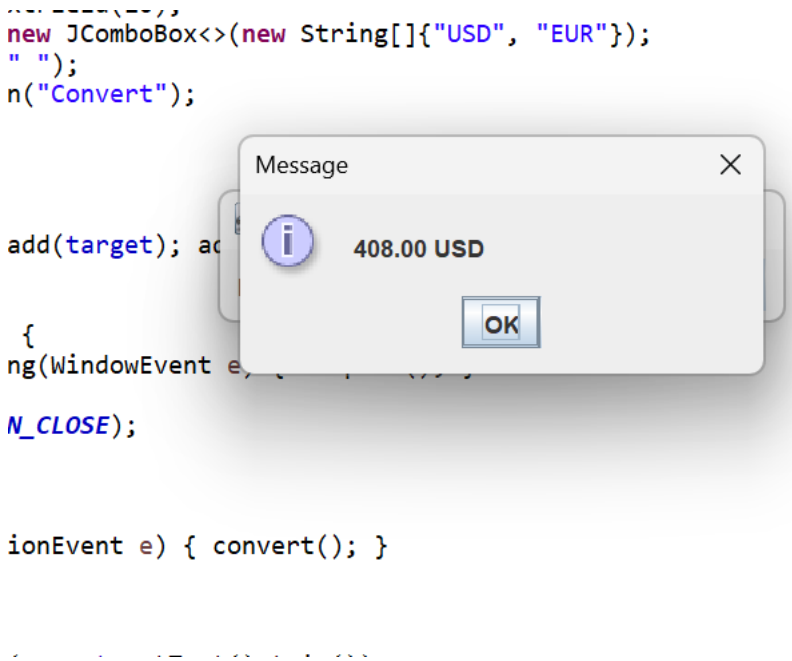
DO_NOTHING_ON_CLOSE);
veTo(null);


erformed(ActionEvent e) { convert(); }



.parseDouble(amount.getText().trim());
value) || value < 0) throw new NumberFormatException();
arget.getSelectedItem();
```





(b) Named Inner Class

Java Source Code – ConverterInnerClass.java

File name: ConverterInnerClass.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ConverterInnerClass extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");

    public ConverterInnerClass() {
        super("Converter – Named Inner Class");
        setLayout(new FlowLayout());
        JButton button = new JButton("Convert");
        add(new JLabel("INR")); add(amount); add(target); add(button); add(result);
        ConvertHandler handler = new ConvertHandler();
        button.addActionListener(handler);
        amount.addActionListener(handler);
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        pack(); setLocationRelativeTo(null);
    }
```

```
    private class ConvertHandler implements ActionListener {
        @Override public void actionPerformed(ActionEvent e) {
            try {
                double value = Double.parseDouble(amount.getText().trim());
                if (!Double.isFinite(value) || value < 0) throw new NumberFormatException();
                String c = (String) target.getSelectedItem();
                double rate = c.equals("USD") ? 0.012 : 0.011;
                String answer = String.format("%.2f %s", value * rate, c);
                result.setText(answer);
                JOptionPane.showMessageDialog(ConverterInnerClass.this, answer);
            } catch (NumberFormatException ex) {
                result.setText("Invalid amount");
                JOptionPane.showMessageDialog(ConverterInnerClass.this, "Invalid amount");
            }
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ConverterInnerClass().setVisible(true));
    }
}
```

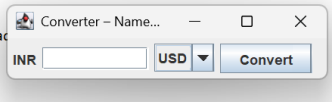
Assignment3/src/q7/ParallelStreamDemo.java

```
package q4;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

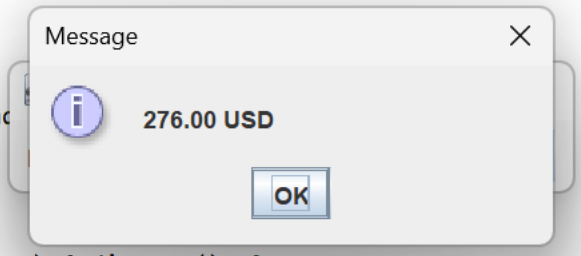
public class ConverterInnerClass extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");

    public ConverterInnerClass() {
        super("Converter - Named Inner Class");
        setLayout(new FlowLayout());
        JButton button = new JButton("Convert");
        add(new JLabel("INR")); add(amount); add(target); ad
        ConvertHandler handler = new ConvertHandler();
        button.addActionListener(handler);
        amount.addActionListener(handler);
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        pack(); setLocationRelativeTo(null);
    }

    private class ConvertHandler implements ActionListener {
        @Override public void actionPerformed(ActionEvent e) {
            try {
                double value = Double.parseDouble(amount.getText().trim());
                if (!Double.isFinite(value) || value < 0) throw new NumberFormatException();
                String c = (String) target.getSelectedItem();
                double rate = c.equals("USD") ? 0.012 : 0.011;
                String answer = String.format("%.2f %s", value * rate, c);
                result.setText(answer);
```



```
w JComboBox<>(new String[]{"USD", "EUR"});
");
```




```
;
d(target); ac
dler();

(WindowEvent e) { dispose(); }

CLOSE);

ionListener {
ctionEvent e) {

le(amount.getText().trim());
value < 0) throw new NumberFormatException();
```

(c) Anonymous Inner Class

Java Source Code – ConverterAnonymousClass.java
File name: ConverterAnonymousClass.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ConverterAnonymousClass extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");

    public ConverterAnonymousClass() {
        super("Converter – Anonymous Inner Class");
        setLayout(new FlowLayout());
        JButton button = new JButton("Convert");
        add(new JLabel("INR")); add(amount); add(target); add(button); add(result);

        ActionListener handler = new ActionListener() {
            @Override public void actionPerformed(ActionEvent e) {
                try {
                    double value = Double.parseDouble(amount.getText().trim());
                    if (!Double.isFinite(value) || value < 0) throw new NumberFormatException();
                    String c = (String) target.getSelectedItem();
                    double rate = c.equals("USD") ? 0.012 : 0.011;
                    String answer = String.format("%.2f %s", value * rate, c);
```

```
            result.setText(answer);
            JOptionPane.showMessageDialog(ConverterAnonymousClass.this, answer);
          } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
            JOptionPane.showMessageDialog(ConverterAnonymousClass.this, "Invalid amount");
          }
        }
      };
      button.addActionListener(handler);
      amount.addActionListener(handler);
      addWindowListener(new WindowAdapter() {
        @Override public void windowClosing(WindowEvent e) { dispose(); }
      });
      setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
      pack(); setLocationRelativeTo(null);
    }

    public static void main(String[] args) {
      SwingUtilities.invokeLater(() -> new ConverterAnonymousClass().setVisible(true));
    }
}
```

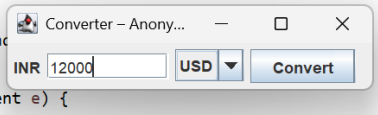
```
package q4;

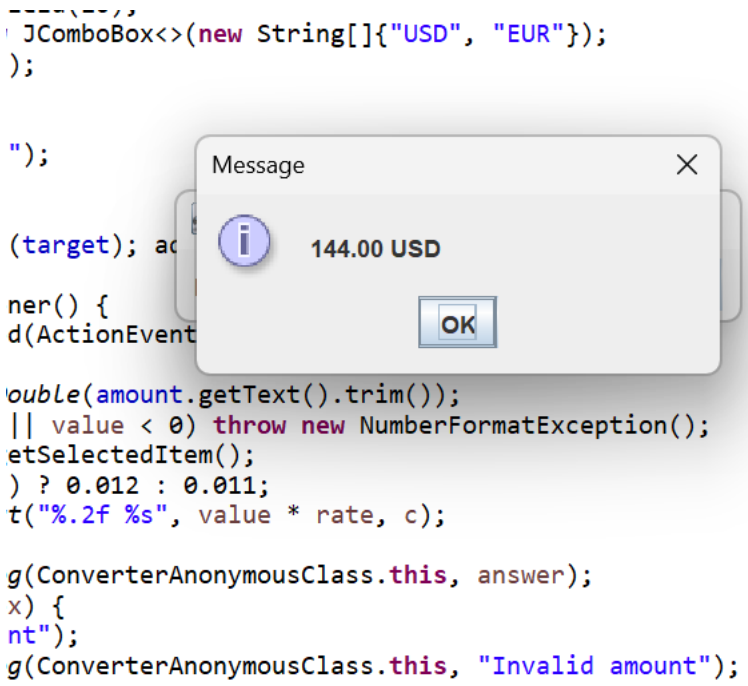
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ConverterAnonymousClass extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");

    public ConverterAnonymousClass() {
        super("Converter – Anonymous Inner Class");
        setLayout(new FlowLayout());
        JButton button = new JButton("Convert");
        add(new JLabel("INR")); add(amount); add(target); ad

        ActionListener handler = new ActionListener() {
            @Override public void actionPerformed(ActionEvent e) {
                try {
                    double value = Double.parseDouble(amount.getText().trim());
                    if (!Double.isFinite(value) || value < 0) throw new NumberFormatException();
                    String c = (String) target.getSelectedItem();
                    double rate = c.equals("USD") ? 0.012 : 0.011;
                    String answer = String.format("%.2f %s", value * rate, c);
                    result.setText(answer);
                    JOptionPane.showMessageDialog(ConverterAnonymousClass.this, answer);
                } catch (NumberFormatException ex) {
                    result.setText("Invalid amount");
                    JOptionPane.showMessageDialog(ConverterAnonymousClass.this, "Invalid amount");
                }
            }
        };
        button.addActionListener(handler);
```





(d) Lambda Expression – Completed TODO Program

Java Source Code – ConverterLambda.java

File name: ConverterLambda.java

```
import java.awt.FlowLayout;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;

public class ConverterLambda extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target =
        new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");

    private static double rate(String currency) {
        return currency.equals("USD") ? 0.012 : 0.011;
    }

    public ConverterLambda() {
```

```
    super("Currency Converter");
    setLayout(new FlowLayout());
    JButton convertButton = new JButton("Convert");
    add(new JLabel("INR"));
    add(amount);
    add(target);
    add(convertButton);
    add(result);

    // One lambda listener is shared by the button and Enter key.
    java.awt.event.ActionListener convertAction = e -> convert();
    convertButton.addActionListener(convertAction);
    amount.addActionListener(convertAction);

    setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
    addWindowListener(new WindowAdapter() {
        @Override public void windowClosing(WindowEvent e) {
            dispose();
        }
    });
    pack();
    setLocationRelativeTo(null);
}

private void convert() {
    try {
        double inr = Double.parseDouble(amount.getText().trim());
        if (!Double.isFinite(inr) || inr < 0) {
            throw new NumberFormatException();
        }
        String currency = (String) target.getSelectedItem();
        double converted = inr * rate(currency);
        String message = String.format("%.2f %s", converted, currency);
        result.setText(message);
        JOptionPane.showMessageDialog(this, message, "Conversion result",
            JOptionPane.INFORMATION_MESSAGE);
    } catch (NumberFormatException ex) {
        result.setText("Invalid amount");
        JOptionPane.showMessageDialog(this, "Invalid amount. Enter a non-negative number.",
            "Input error", JOptionPane.ERROR_MESSAGE);
    }
}

public static void main(String[] args) {
```

```
        SwingUtilities.invokeLater(() -> new ConverterLambda().setVisible(true));
    }
}
```

```
package q4;

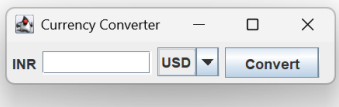
import java.awt.FlowLayout;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;

public class ConverterLambda extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target =
        new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");

    private static double rate(String currency) {
        return currency.equals("USD") ? 0.012 : 0.011;
    }

    public ConverterLambda() {
        super("Currency Converter");
        setLayout(new FlowLayout());
        JButton convertButton = new JButton("Convert");
        add(new JLabel("INR"));
        add(amount);
        add(target);
        add(convertButton);
        add(result);

        // One lambda listener is shared by the button and Enter key.
```



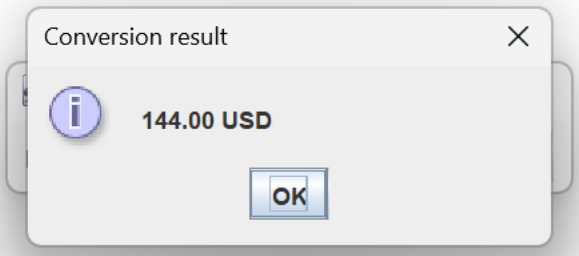

```
ld(10);

);


011;



rt");
```



5.2 Change the Requirement

To support the Enter key, the same conversion handler is registered with the amount text field using amount.addActionListener(...). To display the result in a dialog, JOptionPane.showMessageDialog(...) is called after successful conversion. On invalid input, the program displays an error dialog and updates the result label.

Implementation style	Change for Enter key	Change for result dialog
Frame implements ActionListener	Register the frame as listener on the amount field as well as the button.	Call JOptionPane.showMessageDialog() in the action handler or a shared conversion method.
Named inner class	Register the same named handler instance with the amount field.	Add the dialog call in the inner handler or delegate to a shared method.
Anonymous inner class	Keep the anonymous listener in a variable and register it with both components.	Add the dialog call in the anonymous listener callback.
Lambda expression	Store the lambda in one ActionListener variable and register it with both components.	Add the dialog call in the shared conversion method.

5.3 Compare the Styles

Style	Readability	Access and reuse	Evaluation
Frame implements ActionListener	Straightforward for a small window, but can make the frame responsible for too much.	Direct access to frame fields; one frame listener can handle several controls but may need source checks.	Less suitable for a large application because UI and event-handling responsibilities become tightly coupled.
Named inner class	More lines, but the handler has a meaningful name.	Direct access to enclosing fields; can be reused within the enclosing class.	Useful when the handler has substantial logic or deserves a descriptive name.
Anonymous inner class	Keeps the handler near the component registration but can be verbose.	Can access enclosing fields; usually created for a particular use.	Useful for one-off handlers, although large callbacks reduce readability.
Lambda expression	Concise and clear for short event actions.	Can access enclosing fields; a single listener variable can be registered on multiple controls.	Suitable for short actions. Complex logic should be moved into a named method or separate class.

For a large application, I would keep substantial operations in controller or service methods and use small lambdas or method references for event wiring. This keeps the frame focused on presentation, makes business logic easier to test and avoids duplicating the same operation in several listeners.

Alternative Concept and Complexity Analysis

A method reference such as this::convert provides a concise callback when the method signature matches the listener interface. A shared Action object is useful when the same operation must be available from a button, menu item and keyboard shortcut. The shared Action can centralise the operation and related presentation properties. Reusing one listener or Action avoids writing separate conversion logic for each trigger. The exact allocation behaviour of lambda expressions and method references depends on compiler and runtime implementation; the important design advantage is reuse of one logical action.

The assignment's supplied lambda program is included as ConverterLambda.java. It registers one ActionListener with both the Convert button and the amount text field. The convert() method parses the amount, rejects invalid or negative values, formats the converted result to two decimal places and displays an error dialog for invalid input.

TASK 6 – DESIGN CHALLENGE (13 MARKS)

Campus Event Registration Desktop Application

The campus cultural committee requires a desktop application through which students can register for events and committee members can view, filter and export registration records. The data is stored in MySQL. The application must validate input, prevent duplicate registration for the same event and remain responsive while database operations are in progress.

6.1 Stage 1 – Interpret the Problem

Category	Identified requirements
Users	Students submit event registrations. Committee members view, filter and export registrations.
Screens	Registration form, registration table and filter area, confirmation/error dialogs and an About dialog.
Data	Registration ID, student ID, student name, event name, email address and registration timestamp.
Validation rules	Student ID, name, event and email must be valid and present.
Duplicate rule	A student may not register more than once for the same event.
Responsiveness	Database insertion and retrieval must run outside the Swing Event Dispatch Thread.
Persistence	MySQL stores registration records and enforces uniqueness of the student/event pair.

Testable Requirements

- TR1: A valid new registration is stored and a success message is displayed.
- TR2: An empty or malformed student ID is rejected and no record is inserted.
- TR3: An invalid email address is rejected.
- TR4: A second registration by the same student for the same event is rejected.
- TR5: The same student can register for a different event if the other fields are valid.
- TR6: Selecting an event filter displays only registrations for that event.
- TR7: Export CSV creates a file containing the records currently displayed in the table.
- TR8: The window remains responsive while database work is executing.

6.2 Stage 2 – Propose the Screen and Event Flow

Main JFrame – Campus Event Registration

JMenuBar

File: Refresh, Exit

Help: About

- Registration JPanel
  - Student ID JTextField
  - Student Name JTextField
  - Event JComboBox
  - Email JTextField
  - Register JButton
  - Clear JButton
- Registration Table JPanel
  - Event filter JComboBox
  - Apply Filter JButton
  - Export CSV JButton
  - Previous JButton
  - Next JButton
  - JTable inside JScrollPane
- Status JLabel

Registration event flow: Open application → enter student details → click Register → validate input → check duplicate → insert record → display success/error → refresh table. Committee event flow: choose event → apply filter → retrieve matching page → display records → export displayed records to CSV.

6.3 Stage 3 – Critical Design Decisions

Input validation is implemented in RegistrationService so that the rules remain independent of the user interface. RegistrationDAO contains the SQL statements and uses PreparedStatement to bind user-supplied values. Duplicate checking is performed by the service through the DAO, while a UNIQUE constraint on (student_id, event_name) in MySQL provides the final protection against simultaneous duplicate requests. Long-running database insertion and retrieval are executed in SwingWorker.doInBackground(). The done() method updates the Swing interface on the Event Dispatch Thread. Registration records are retrieved in pages using LIMIT and OFFSET.

6.4 Stage 4 – Class Design and Handlers

Class relationship:

EventRegistrationApp (JFrame) → RegistrationService → RegistrationDAO → DBConnection → MySQL

Registration is the data model used to transfer registration details from the database layer to the presentation layer.

Algorithm: REGISTER_ACTION

19. Read the student ID, student name, event and email from the registration form.
20. Set the status label to indicate that registration is being processed.
21. Start a SwingWorker task.

22. In doInBackground(), call RegistrationService.register(...).
23. Validate the student ID, student name, selected event and email.
24. Check whether the same student ID and event already exist.
25. If a duplicate exists, reject the request with a clear message.
26. Otherwise, insert the record using PreparedStatement. The database UNIQUE constraint provides an additional duplicate check.
27. In done(), display the success or error message. On success, clear the form and refresh the table.
28. Terminate the algorithm.

Algorithm: FILTER_ACTION

29. Read the selected event filter.
30. Reset the page number when a new filter is applied.
31. Display a loading status and start a SwingWorker task.
32. In doInBackground(), call the service method to retrieve the selected page.
33. The DAO executes a SELECT query with an optional event condition, LIMIT and OFFSET.
34. In done(), clear the existing table rows and insert the returned records.
35. Update the status label with the page number and number of records loaded.
36. Terminate the algorithm.

6.5 Stage 5 – Implementation

The implementation is divided into a presentation layer, service layer, data-access layer and database connection utility. The SQL schema creates the campus_events database and registrations table. The UNIQUE constraint prevents a student from registering twice for the same event.

SQL Script – schema.sql

```
CREATE DATABASE IF NOT EXISTS campus_events;
USE campus_events;

CREATE TABLE IF NOT EXISTS registrations (
    id INT PRIMARY KEY AUTO_INCREMENT,
    student_id VARCHAR(20) NOT NULL,
    student_name VARCHAR(100) NOT NULL,
    event_name VARCHAR(80) NOT NULL,
    email VARCHAR(150) NOT NULL,
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_student_event UNIQUE (student_id, event_name)
);

-- The UNIQUE constraint is the final database-level protection against duplicate
-- registration, even if two requests arrive almost simultaneously.
```

Java Source Code – DBConnection.java

Java Source Code – DBConnection.java

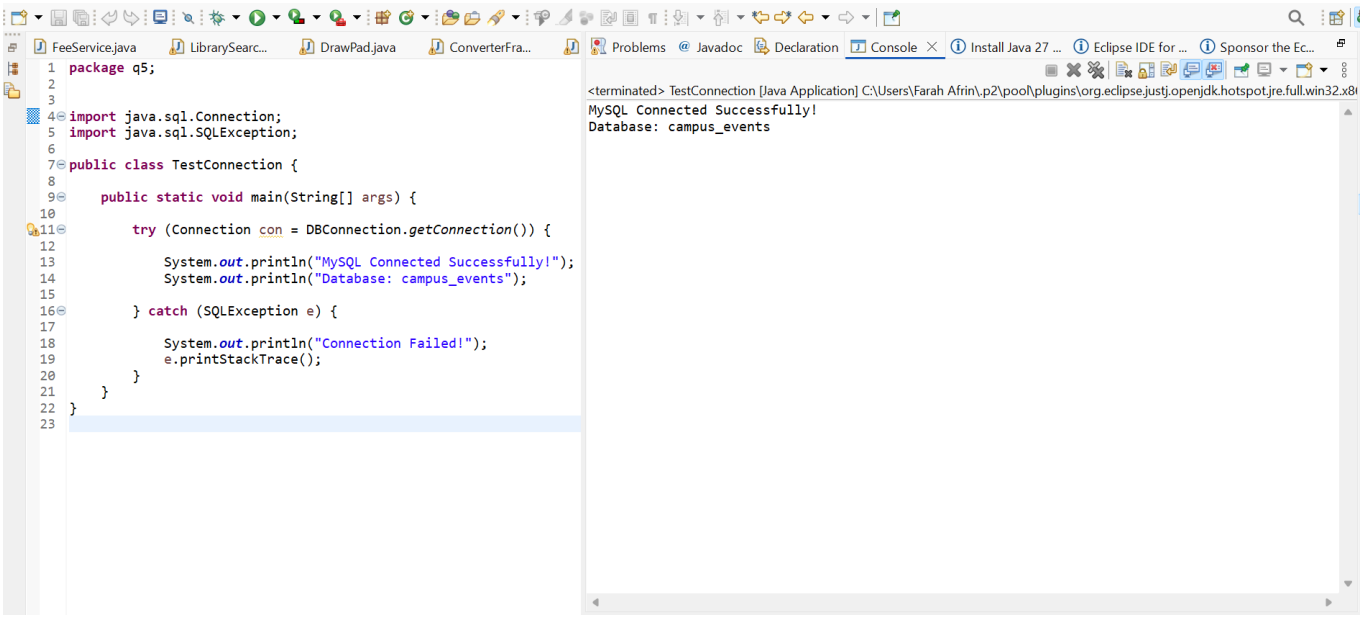
File name: DBConnection.java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public final class DBConnection {
    private static final String URL =
        "jdbc:mysql://localhost:3306/campus_events?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD = "YOUR_MYSQL_PASSWORD";

    private DBConnection() { }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```




```
package q5;


import java.sql.Connection;
import java.sql.SQLException;

public class TestConnection {

    public static void main(String[] args) {

        try (Connection con = DBConnection.getConnection()) {

            System.out.println("MySQL Connected Successfully!");
            System.out.println("Database: campus_events");

        } catch (SQLException e) {

            System.out.println("Connection Failed!");
            e.printStackTrace();
        }
    }
}
```

Java Source Code – Registration.java

Java Source Code – Registration.java

File name: Registration.java

```
public class Registration {
    private final int id;
    private final String studentId;
    private final String studentName;
    private final String eventName;
    private final String email;
```

```
    public Registration(int id, String studentId, String studentName,
                        String eventName, String email) {
        this.id = id;
        this.studentId = studentId;
        this.studentName = studentName;
        this.eventName = eventName;
        this.email = email;
    }

    public int getId() { return id; }
    public String getStudentId() { return studentId; }
    public String getStudentName() { return studentName; }
    public String getEventName() { return eventName; }
    public String getEmail() { return email; }
}
```

Java Source Code – RegistrationDAO.java

Java Source Code – RegistrationDAO.java

File name: RegistrationDAO.java

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/** JDBC data-access layer: SQL is isolated from the user interface. */
public class RegistrationDAO {
    public void insert(Registration r) throws SQLException {
        String sql = "INSERT INTO registrations(student_id, student_name, event_name, email) "
                   + "VALUES (?, ?, ?, ?)";
        try (Connection c = DBConnection.getConnection();
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, r.getStudentId());
            ps.setString(2, r.getStudentName());
            ps.setString(3, r.getEventName());
            ps.setString(4, r.getEmail());
            ps.executeUpdate();
        }
    }

    public List<Registration> findPage(String event, int limit, int offset)
            throws SQLException {
        List<Registration> rows = new ArrayList<>();
        boolean all = event == null || event.equals("All Events");
        String sql = "SELECT id, student_id, student_name, event_name, email "
```

```
            + "FROM registrations " + (all ? "" : "WHERE event_name = ? ")
            + "ORDER BY id DESC LIMIT ? OFFSET ?";
    try (Connection c = DBConnection.getConnection();
         PreparedStatement ps = c.prepareStatement(sql)) {
        int index = 1;
        if (!all) ps.setString(index++, event);
        ps.setInt(index++, limit);
        ps.setInt(index, offset);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                rows.add(new Registration(rs.getInt("id"),
                    rs.getString("student_id"), rs.getString("student_name"),
                    rs.getString("event_name"), rs.getString("email")));
            }
        }
    }
    return rows;
}

public List<Registration> findAll(String event) throws SQLException {
    List<Registration> rows = new ArrayList<>();
    boolean all = event == null || event.equals("All Events");
    String sql = "SELECT id, student_id, student_name, event_name, email "
            + "FROM registrations " + (all ? "" : "WHERE event_name = ? ")
            + "ORDER BY id DESC";
    try (Connection c = DBConnection.getConnection();
         PreparedStatement ps = c.prepareStatement(sql)) {
        if (!all) ps.setString(1, event);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                rows.add(new Registration(rs.getInt("id"),
                    rs.getString("student_id"), rs.getString("student_name"),
                    rs.getString("event_name"), rs.getString("email")));
            }
        }
    }
    return rows;
}

public boolean exists(String studentId, String eventName) throws SQLException {
    String sql = "SELECT 1 FROM registrations WHERE student_id = ? AND event_name = ?";
    try (Connection c = DBConnection.getConnection();
         PreparedStatement ps = c.prepareStatement(sql)) {
        ps.setString(1, studentId);
```

```
            ps.setString(2, eventName);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next();
            }
        }
    }
}
```

Java Source Code – RegistrationService.java

Java Source Code – RegistrationService.java

File name: RegistrationService.java

```
import java.sql.SQLException;
import java.util.List;

public class RegistrationService {
    private final RegistrationDAO dao = new RegistrationDAO();

    public void register(String studentId, String name, String event, String email)
            throws SQLException {
        if (studentId == null || !studentId.trim().matches("[A-Za-z0-9-]{3,20}")) {
            throw new IllegalArgumentException("Student ID must contain 3–20 letters, digits or
hyphens.");
        }
        if (name == null || name.trim().length() < 2) {
            throw new IllegalArgumentException("Enter a valid student name.");
        }
        if (event == null || event.trim().isEmpty() || event.equals("Select Event")) {
            throw new IllegalArgumentException("Choose an event.");
        }
        if (email == null || !email.trim().matches("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-
z]{2,}$")) {
            throw new IllegalArgumentException("Enter a valid email address.");
        }
        if (dao.exists(studentId.trim(), event)) {
            throw new IllegalArgumentException("This student is already registered for this event.");
        }
        dao.insert(new Registration(0, studentId.trim(), name.trim(), event, email.trim()));
    }

    public List<Registration> getAll(String event) throws SQLException {
        return dao.findAll(event);
    }
```

```
    public List<Registration> getPage(String event, int page, int pageSize)
            throws SQLException {
        return dao.findPage(event, pageSize, page * pageSize);
    }
}
```

Java Source Code – EventRegistrationApp.java

Java Source Code – EventRegistrationApp.java

File name: EventRegistrationApp.java

```
import java.awt.*;
import java.awt.event.*;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.List;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

/**
 * Swing presentation layer. Database tasks run in SwingWorker so the EDT remains responsive.
 */
public class EventRegistrationApp extends JFrame {
    private static final int PAGE_SIZE = 20;
    private final RegistrationService service = new RegistrationService();

    private final JTextField studentId = new JTextField(12);
    private final JTextField studentName = new JTextField(12);
    private final JTextField email = new JTextField(15);
    private final JComboBox<String> event = new JComboBox<>(new String[]{
        "Select Event", "TechFest", "Cultural Fest", "Sports Meet", "Workshop"
    });
    private final JComboBox<String> filter = new JComboBox<>(new String[]{
        "All Events", "TechFest", "Cultural Fest", "Sports Meet", "Workshop"
    });
    private final DefaultTableModel model = new DefaultTableModel(
        new String[]{"ID", "Student ID", "Student Name", "Event", "Email"}, 0) {
        @Override public boolean isCellEditable(int row, int column) { return false; }
    };
    private final JTable table = new JTable(model);
    private final JLabel status = new JLabel("Ready");
    private int currentPage = 0;

    public EventRegistrationApp() {
```

```
super("Campus Event Registration System");
setLayout(new BorderLayout(10, 10));
setJMenuBar(createMenuBar());

JLabel heading = new JLabel("CAMPUS EVENT REGISTRATION", SwingConstants.CENTER);
heading.setFont(new Font("SansSerif", Font.BOLD, 20));
heading.setBorder(BorderFactory.createEmptyBorder(12, 5, 8, 5));
add(heading, BorderLayout.NORTH);

JPanel form = new JPanel(new GridLayout(5, 2, 8, 8));
form.setBorder(BorderFactory.createTitledBorder("Student Registration"));
form.add(new JLabel("Student ID:")); form.add(studentId);
form.add(new JLabel("Student name:")); form.add(studentName);
form.add(new JLabel("Event:")); form.add(event);
form.add(new JLabel("Email:")); form.add(email);
JButton register = new JButton("Register");
JButton clear = new JButton("Clear");
form.add(register); form.add(clear);

JPanel tablePanel = new JPanel(new BorderLayout(5, 5));
JPanel filterPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
filterPanel.add(new JLabel("Filter by event:"));
filterPanel.add(filter);
JButton applyFilter = new JButton("Apply Filter");
JButton export = new JButton("Export CSV");
JButton previous = new JButton("Previous");
JButton next = new JButton("Next");
filterPanel.add(applyFilter);
filterPanel.add(export);
filterPanel.add(previous);
filterPanel.add(next);
tablePanel.add(filterPanel, BorderLayout.NORTH);
table.setFillsViewportHeight(true);
tablePanel.add(new JScrollPane(table), BorderLayout.CENTER);

JPanel center = new JPanel(new BorderLayout(10, 10));
center.setBorder(BorderFactory.createEmptyBorder(0, 12, 0, 12));
center.add(form, BorderLayout.NORTH);
center.add(tablePanel, BorderLayout.CENTER);
add(center, BorderLayout.CENTER);
status.setBorder(BorderFactory.createEmptyBorder(7, 12, 7, 12));
add(status, BorderLayout.SOUTH);

register.addActionListener(e -> registerStudent());
```

```
    clear.addActionListener(e -> clearForm());
    applyFilter.addActionListener(e -> { currentPage = 0; loadPage(); });
    previous.addActionListener(e -> {
        if (currentPage > 0) { currentPage--; loadPage(); }
    });
    next.addActionListener(e -> { currentPage++; loadPage(); });
    export.addActionListener(e -> exportCsv());

    setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
    addWindowListener(new WindowAdapter() {
        @Override public void windowClosing(WindowEvent e) {
            int answer = JOptionPane.showConfirmDialog(EventRegistrationApp.this,
                "Exit the registration system?", "Confirm exit",
                JOptionPane.YES_NO_OPTION);
            if (answer == JOptionPane.YES_OPTION) dispose();
        }
    });
    setSize(1000, 650);
    setLocationRelativeTo(null);
    loadPage();
}

private JMenuBar createMenuBar() {
    JMenuBar bar = new JMenuBar();
    JMenu file = new JMenu("File");
    JMenuItem refresh = new JMenuItem("Refresh");
    JMenuItem exit = new JMenuItem("Exit");
    refresh.addActionListener(e -> { currentPage = 0; loadPage(); });
    exit.addActionListener(e -> dispatchEvent(
        new WindowEvent(this, WindowEvent.WINDOW_CLOSING)));
    file.add(refresh); file.addSeparator(); file.add(exit);
    JMenu help = new JMenu("Help");
    JMenuItem about = new JMenuItem("About");
    about.addActionListener(e -> JOptionPane.showMessageDialog(this,
        "Campus Event Registration System\nProgramming in Java – CO4"));
    help.add(about);
    bar.add(file); bar.add(help);
    return bar;
}

private void registerStudent() {
    final String idValue = studentId.getText();
    final String nameValue = studentName.getText();
    final String eventValue = (String) event.getSelectedItem();
```

```
    final String emailValue = email.getText();
    String idValue = studentId.getText();
    String nameValue = studentName.getText();
    String eventValue = (String) event.getSelectedItem();
    String emailValue = email.getText();
    setBusy(true, "Validating and saving registration...");
    new SwingWorker<Void, Void>() {
        @Override protected Void doInBackground() throws Exception {
            service.register(idValue, nameValue, eventValue, emailValue);
            return null;
        }
        @Override protected void done() {
            setBusy(false, "Ready");
            try {
                get();
                JOptionPane.showMessageDialog(EventRegistrationApp.this,
                    "Registration completed successfully.",
                    "Success", JOptionPane.INFORMATION_MESSAGE);
                clearForm();
                currentPage = 0;
                loadPage();
            } catch (Exception ex) {
                showError(rootMessage(ex));
            }
        }
    }.execute();
}

private void loadPage() {
    setBusy(true, "Loading registrations...");
    String selectedEvent = (String) filter.getSelectedItem();
    final int requestedPage = currentPage;
    new SwingWorker<List<Registration>, Void>() {
        @Override protected List<Registration> doInBackground() throws Exception {
            return service.getPage(selectedEvent, requestedPage, PAGE_SIZE);
        }
        @Override protected void done() {
            try {
                List<Registration> rows = get();
                model.setRowCount(0);
                for (Registration r : rows) {
                    model.addRow(new Object[]{r.getId(), r.getStudentId(),
                        r.getStudentName(), r.getEventName(), r.getEmail()});
                }
```

```
            setBusy(false, "Page " + (currentPage + 1) + ": "
                + rows.size() + " registration(s) loaded.");
        } catch (Exception ex) {
            setBusy(false, "Could not load registrations.");
            showError(rootMessage(ex));
        }
      }
   }.execute();
}

private void exportCsv() {
   String selectedEvent = (String) filter.getSelectedItem();
   setBusy(true, "Preparing CSV export...");
   new SwingWorker<List<Registration>, Void>() {
      @Override protected List<Registration> doInBackground() throws Exception {
         return service.getAll(selectedEvent);
      }
      @Override protected void done() {
         setBusy(false, "Ready");
         try {
            List<Registration> rows = get();
            JFileChooser chooser = new JFileChooser();
            chooser.setSelectedFile(new File("event-registrations.csv"));
            if (chooser.showSaveDialog(EventRegistrationApp.this)
                  != JFileChooser.APPROVE_OPTION) return;
            File file = chooser.getSelectedFile();
            try (FileWriter writer = new FileWriter(file)) {
               writer.write("ID,Student ID,Student Name,Event,Email\n");
               for (Registration r : rows) {
                  String[] values = {String.valueOf(r.getId()), r.getStudentId(),
                     r.getStudentName(), r.getEventName(), r.getEmail()};
                  for (int col = 0; col < values.length; col++) {
                     String value = values[col].replace("\"", "\"\"");
                     writer.write("\"" + value + "\"");
                     if (col < values.length - 1) writer.write(",");
                  }
                  writer.write("\n");
               }
            }
            status.setText("Exported " + rows.size() + " filtered registration(s) to "
               + file.getName());
            JOptionPane.showMessageDialog(EventRegistrationApp.this,
               "CSV export completed successfully.");
         } catch (Exception ex) {
```

```
                showError("CSV export failed: " + rootMessage(ex));
            }
        }
    }.execute();
  }

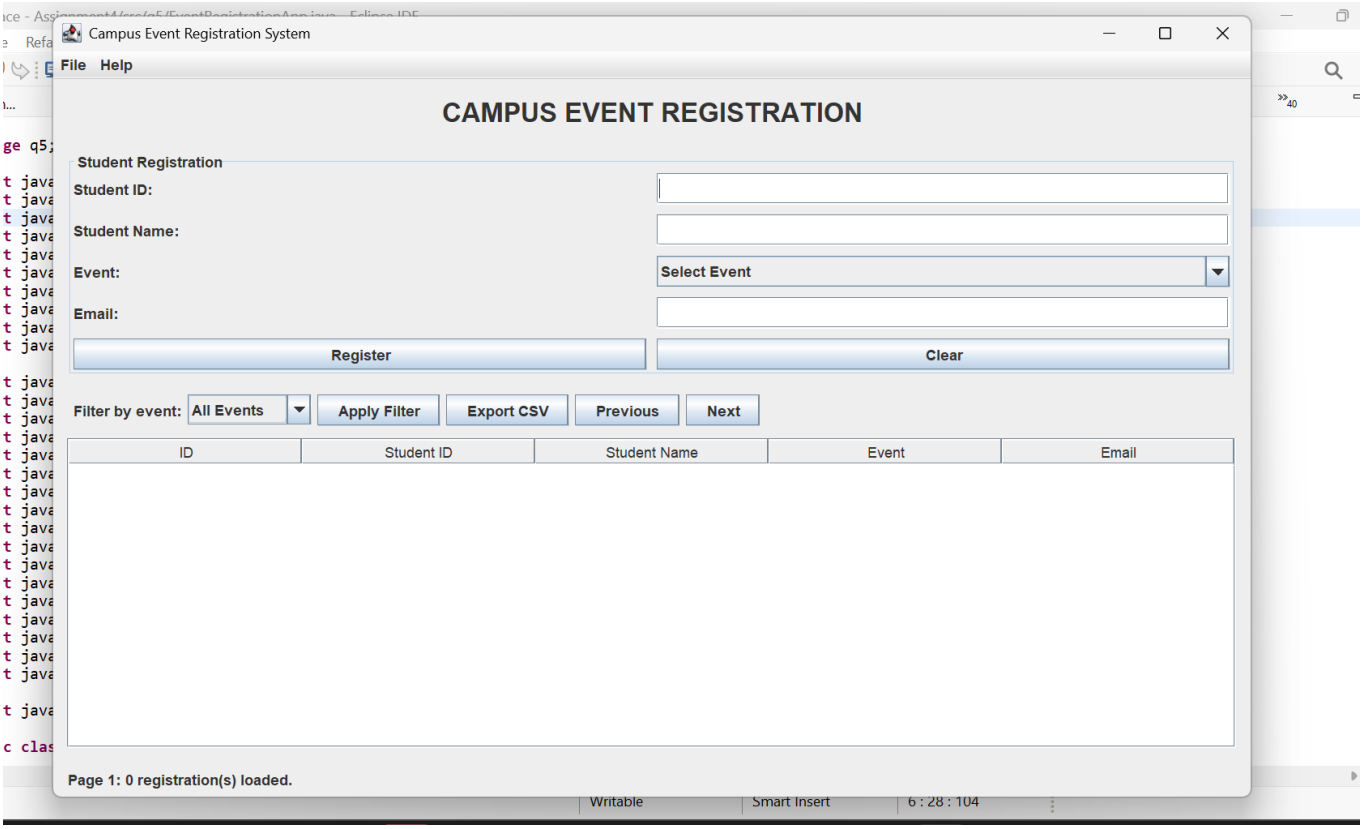
  private void clearForm() {
    studentId.setText("");
    studentName.setText("");
    email.setText("");
    event.setSelectedIndex(0);
    studentId.requestFocus();
  }

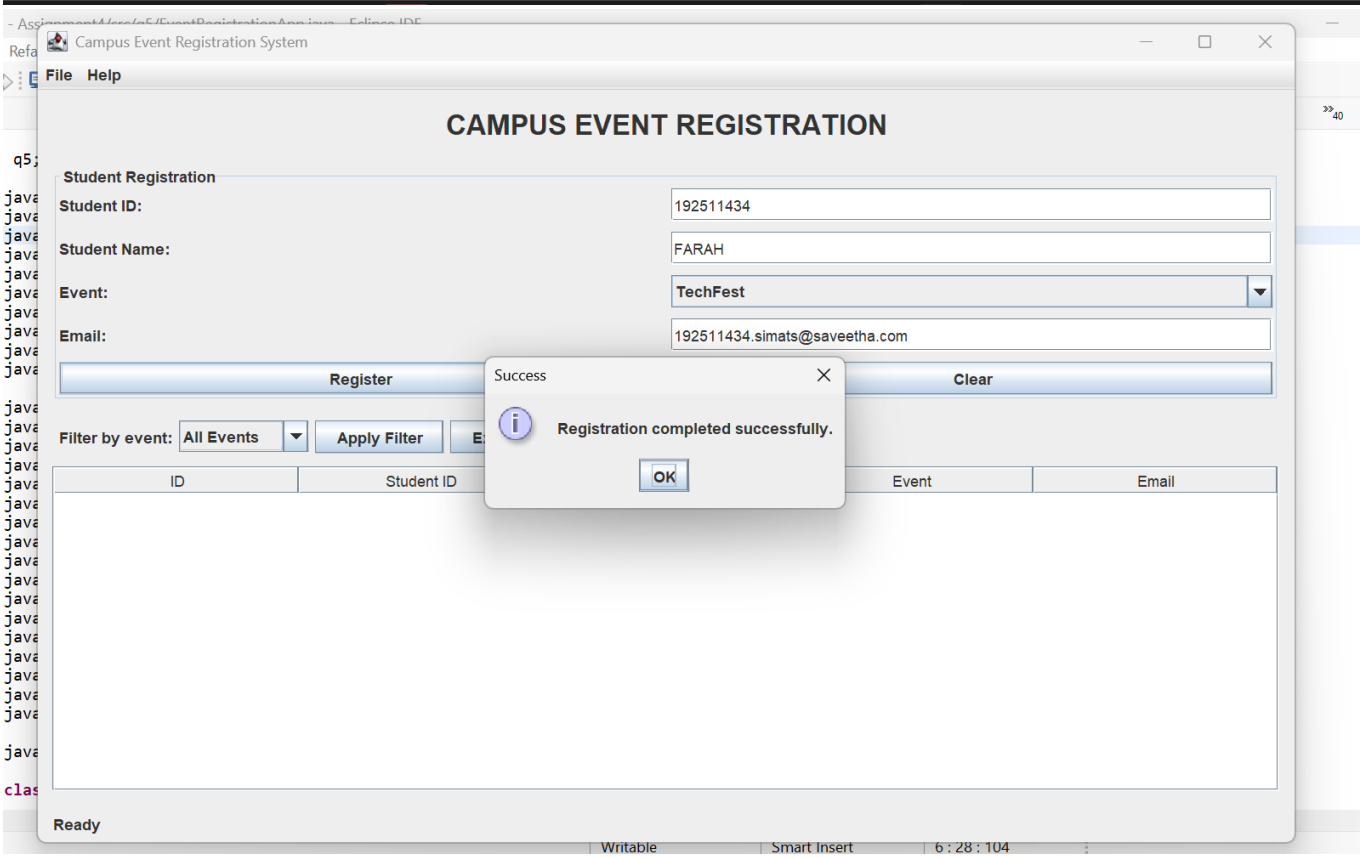
  private void setBusy(boolean busy, String message) {
    status.setText(message);
    setCursor(busy ? Cursor.getPredefinedCursor(Cursor.WAIT_CURSOR)
                   : Cursor.getDefaultCursor());
  }

  private void showError(String message) {
    JOptionPane.showMessageDialog(this, message, "Error",
        JOptionPane.ERROR_MESSAGE);
  }

  private String rootMessage(Exception ex) {
    Throwable cause = ex;
    while (cause.getCause() != null) cause = cause.getCause();
    return cause.getMessage() == null ? ex.toString() : cause.getMessage();
  }

  public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new EventRegistrationApp().setVisible(true));
  }
}
```


Campus Event Registration System
File Help
CAMPUS EVENT REGISTRATION
Student Registration
Student ID:
Student Name:
Event:
Select Event
Email:
Register
Clear
Filter by event:
All Events
Apply Filter
Export CSV
Previous
Next
ID
Student ID
Student Name
Event
Email
Page 1: 0 registration(s) loaded.


Campus Event Registration System
File Help
CAMPUS EVENT REGISTRATION
Student Registration
Student ID:
192511434
Student Name:
FARAH
Event:
TechFest
Email:
192511434.simats@saveetha.com
Register
Clear
Success
Registration completed successfully.
OK
Filter by event:
All Events
Apply Filter
ID
Student ID
Event
Email
Ready

Test Cases and Demonstration Evidence

Test ID	Input / action	Expected result
T6-01	Submit a valid new student ID, name, event and email.	One record is inserted and a success dialog appears.
T6-02	Leave the student ID empty and submit.	The service rejects the input and no record is inserted.
T6-03	Enter an invalid email address.	The email validation message is displayed.
T6-04	Submit the same student ID and event twice.	The second attempt is rejected as a duplicate.
T6-05	Register the same student for a different event.	The registration is accepted if the remaining fields are valid.
T6-06	Select an event in the filter and apply it.	Only matching registrations are displayed.
T6-07	Select Export CSV and save the file.	A CSV file is created containing the displayed page.
T6-08	Load registrations while interacting with the window.	The interface remains responsive while the worker runs.

Alternative Concept and Complexity Analysis

Loading all 100,000 registration records at once requires a transfer and table-model construction proportional to O(N), where N is the total number of records. If the average in-memory size of one displayed record is s, the application-side memory requirement is O(N × s). A large initial load increases network transfer, memory use and garbage-collection activity. If performed on the Event Dispatch Thread, the interface may become unresponsive.

With page size p, the interface receives and stores only O(p) records at a time, giving an approximate table-model memory requirement of O(p × s). The database still performs work to locate the requested page; LIMIT/OFFSET reduces the amount transferred to the application but a large OFFSET may require the database to scan and skip earlier rows. Indexing the filter and ordering columns can improve query performance. SwingWorker keeps the database request off the Event Dispatch Thread, improving interface responsiveness. For very deep pagination, keyset pagination can be more efficient, but this assignment's implementation follows the specified LIMIT/OFFSET approach.

Conclusion

The six tasks demonstrate the analysis, design and implementation of event-driven Java applications. A well-structured GUI requires correct listener registration, suitable layout managers, clear validation and meaningful feedback. Retaining drawing data ensures that graphics survive repainting. Separating the presentation, service and data-access layers improves maintainability and testability. PreparedStatement, database constraints and SwingWorker strengthen the reliability and responsiveness of a GUI–JDBC application.